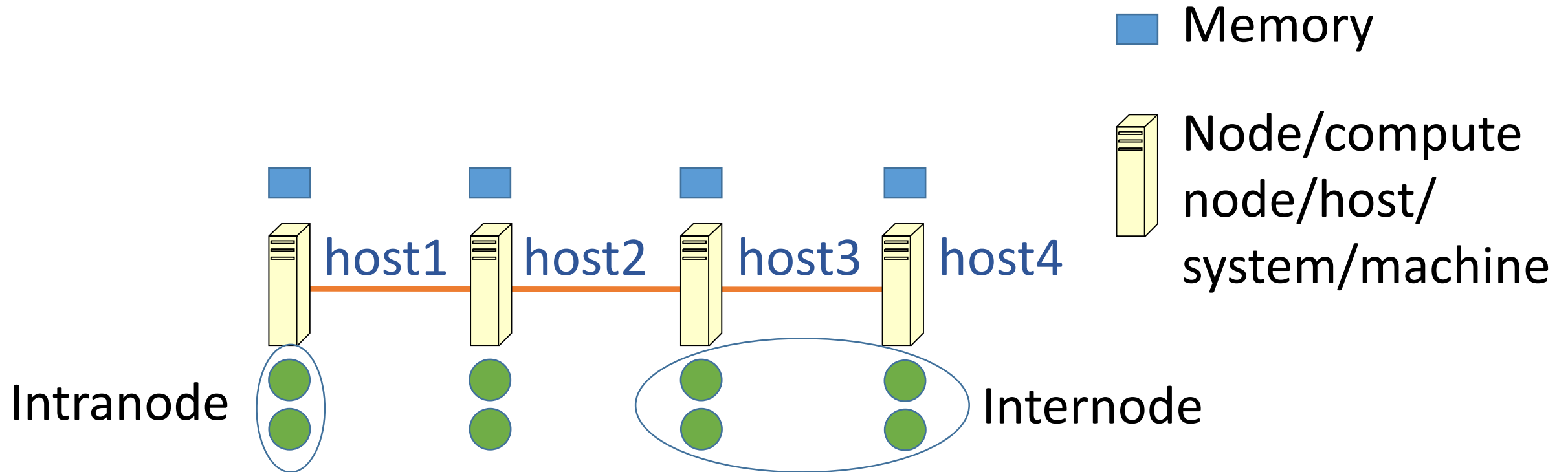


P2P - II

Lecture 5

January 20, 2025

MPI Program Execution



```
mpirun -n 8 -hosts host1,host2,host3,host4 ./exe
```

```
mpirun -n 8 -hosts host1:2,host2:2,host3:2,host4:2 ./exe
```

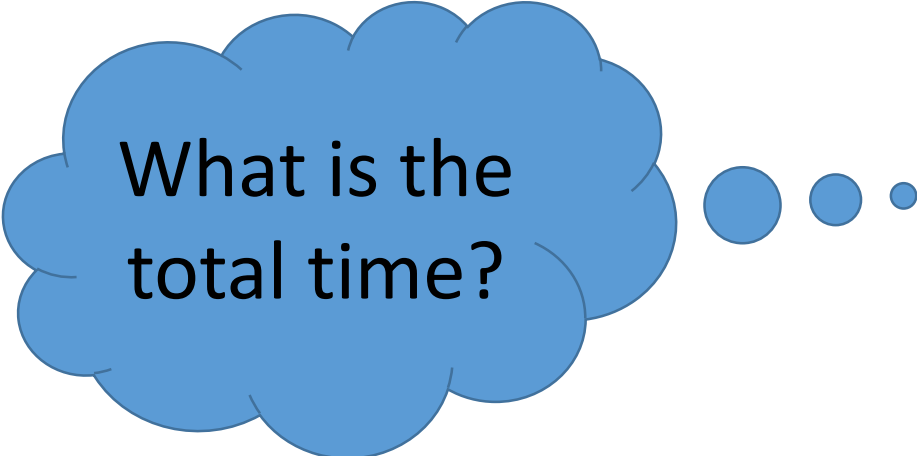
Timing Send/Recv (timingSend.c)

```
MPI_Comm_rank(MPI_COMM_WORLD, &myrank) ;
MPI_Comm_size(MPI_COMM_WORLD, &size);

// initialize data
for (int i=0; i<count; i++)
    buf[i] = myrank+i;

sTime = MPI_Wtime();
if (myrank == 0)
    MPI_Send (buf, count, MPI_INT, 1, 1, MPI_COMM_WORLD);
else
    if (myrank == 1)
        MPI_Recv (buf, count, MPI_INT, 0, 1, MPI_COMM_WORLD, &status);
eTime = MPI_Wtime();
time = eTime - sTime;
```

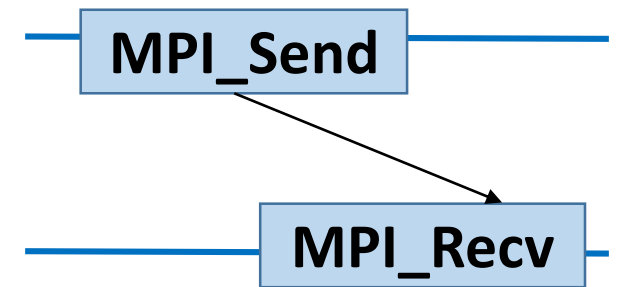
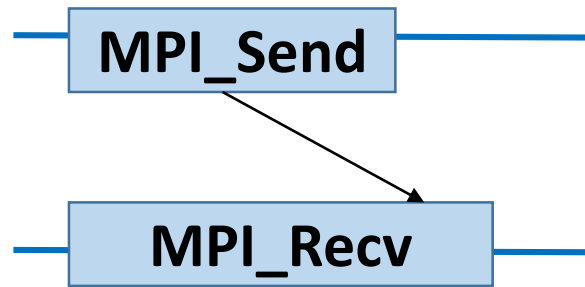
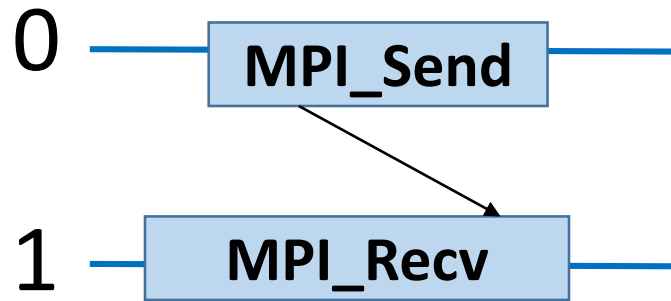
Timing Variation



What is the total time?

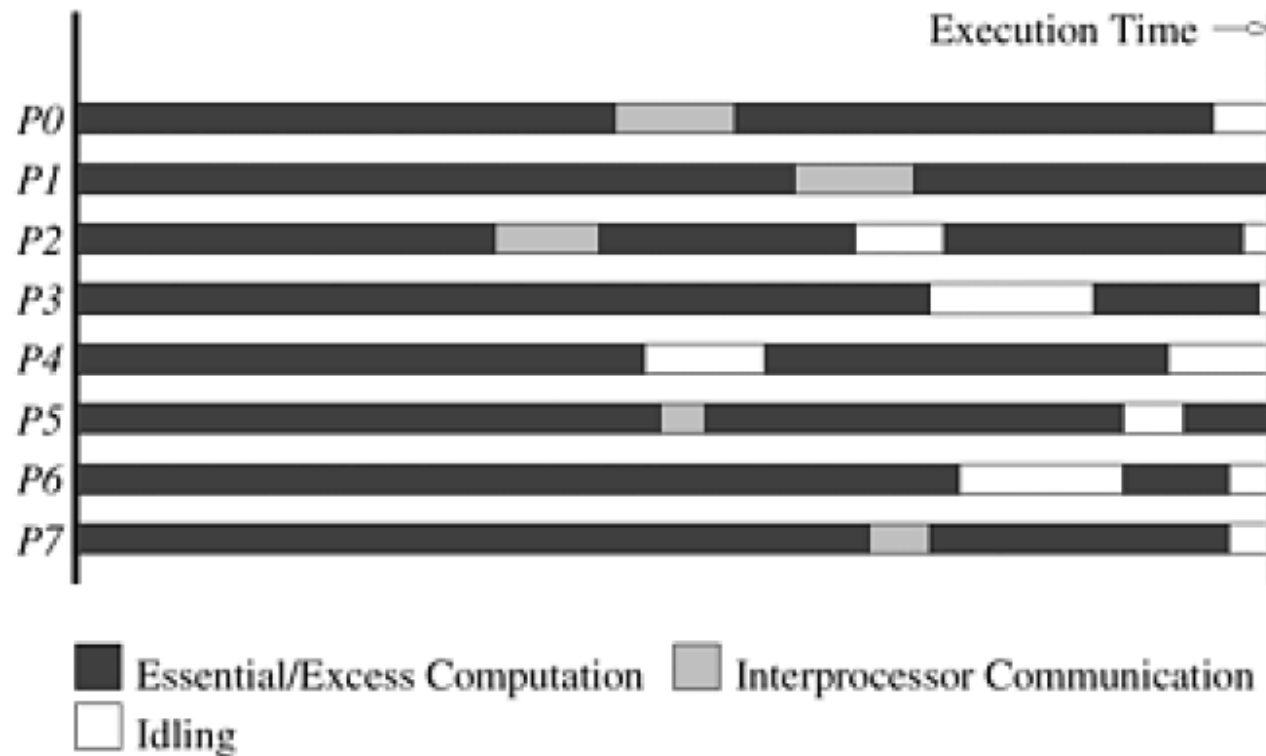
```
class mpirun -np 2 ./timingSend 100
0.000002
0.000039
class mpirun -np 2 ./timingSend 100
0.000002
0.000009
class mpirun -np 2 ./timingSend 10000
0.000022
0.000039
class mpirun -np 2 ./timingSend 10000
0.000022
0.000030
class mpirun -np 2 ./timingSend 1000000
0.000426
0.000477
class mpirun -np 2 ./timingSend 1000000
0.000398
0.000403
class mpirun -np 2 ./timingSend 100000000
0.052907
0.052171
class mpirun -np 2 ./timingSend 100000000
0.055681
0.056406
```

A Few Scenarios



Inspiration: Slides of Prof. Dr. – Ing. Morris Riedel

Execution Profile



```
sTime = MPI_Wtime();  
if (myrank == 0)  
    MPI_Send (buf, count, MPI_INT, 1,  
              1, MPI_COMM_WORLD);  
else if (myrank == 1)  
    MPI_Recv (buf, count, MPI_INT, 0,  
              1, MPI_COMM_WORLD, &status);  
eTime = MPI_Wtime();
```

Execution Profile of a Hypothetical Parallel Program

Source: GGKK Chapter 5

Variation

1. Node to node
latency/bandwidth and
hence communication time
is different and varies

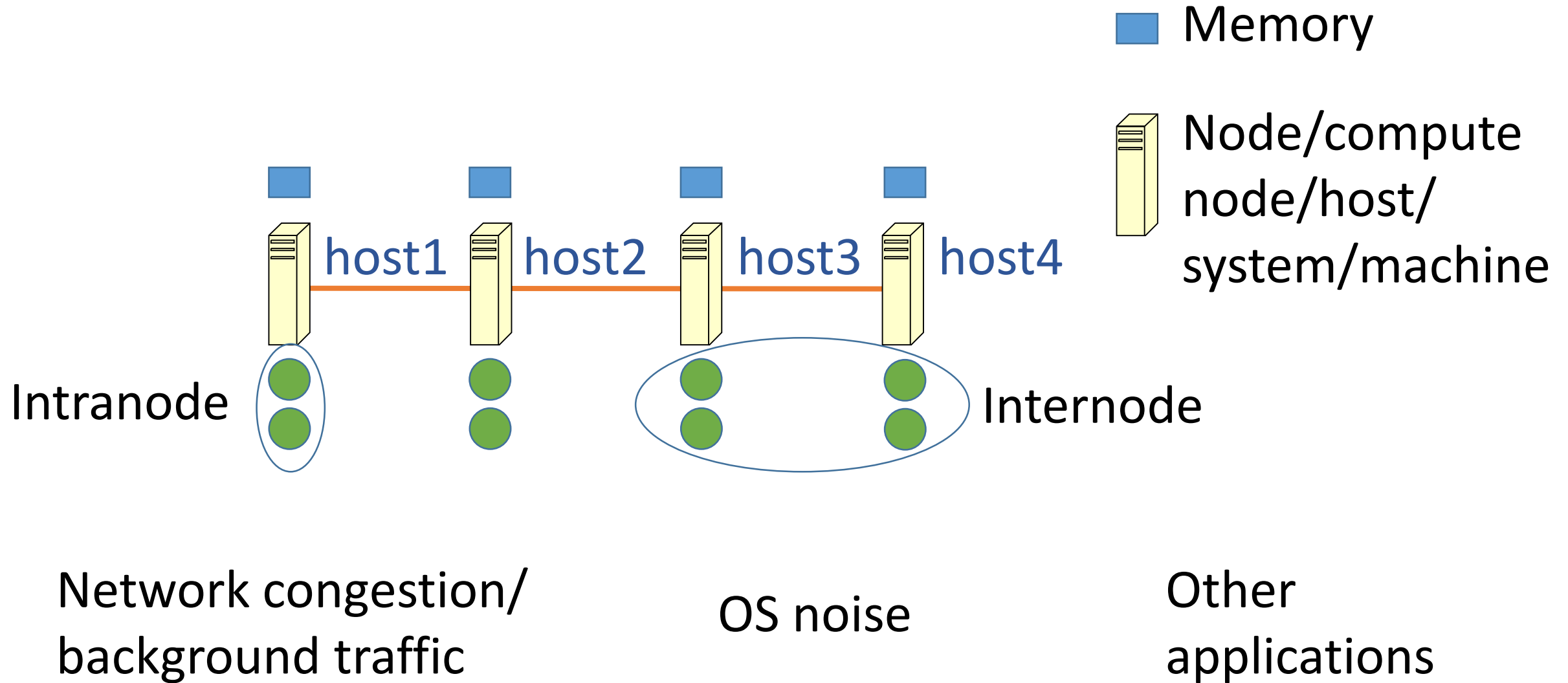
2. Intra-node
communication is typically
faster than inter-node
communication

```
class > mpirun -np 2 ./timingSend 2000
0.000575
0.000634
class > mpirun -np 2 ./timingSend 2000
0.000504
0.000754
class > mpirun -np 2 ./timingSend 2000
0.000568
0.000660
```

```
class >
class >
class > mpirun -np 2 ./timingSend 2000
0.001152
0.000436
class > mpirun -np 2 ./timingSend 2000
0.001096
0.000440
class > mpirun -np 2 ./timingSend 2000
0.001079
0.000420
```

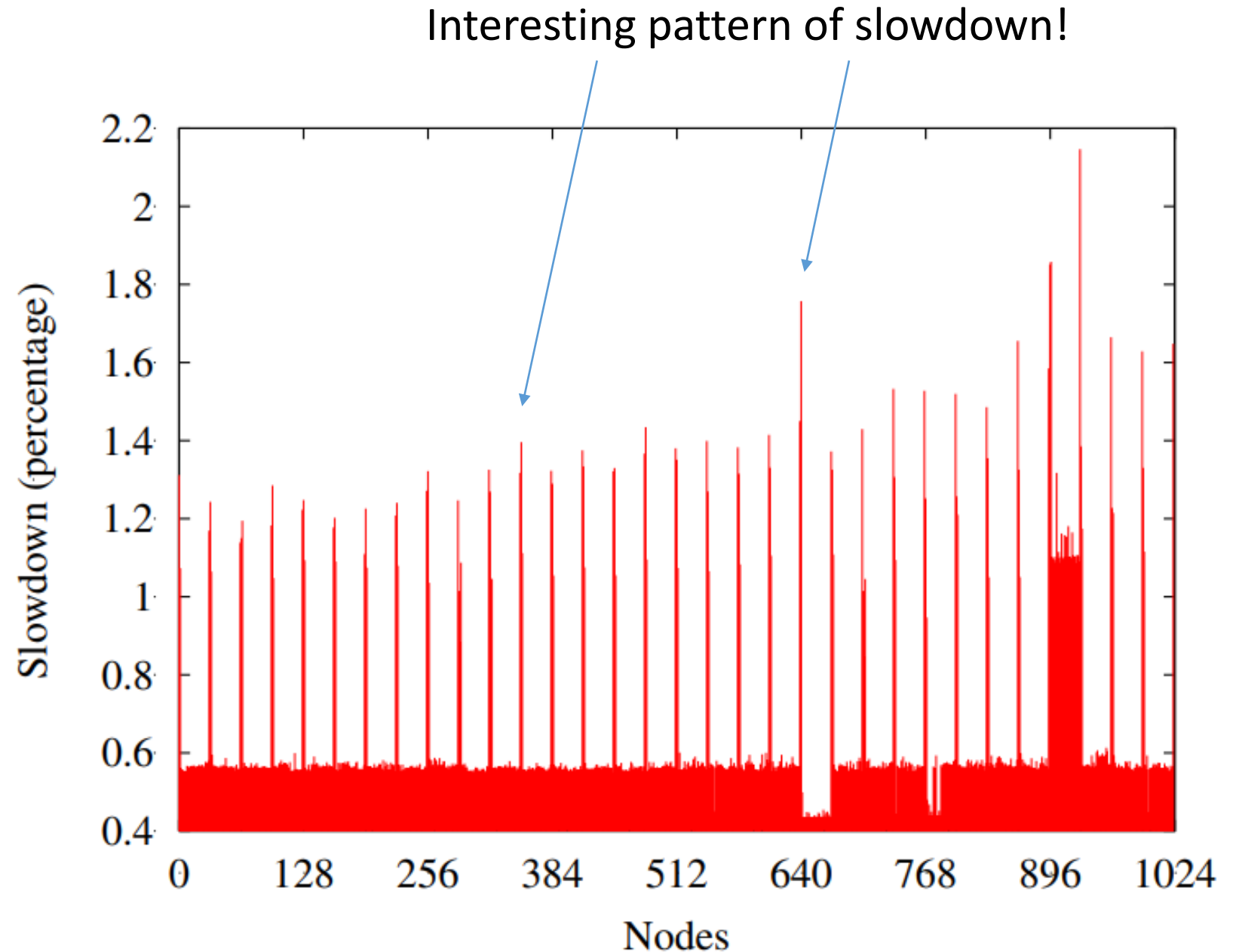
```
class >
class > mpirun -np 2 ./timingSend 2000
0.000008
0.000025
class > mpirun -np 2 ./timingSend 2000
0.000010
0.000010
class > mpirun -np 2 ./timingSend 2000
0.000009
0.000014
```

Sources of Variation



Variability

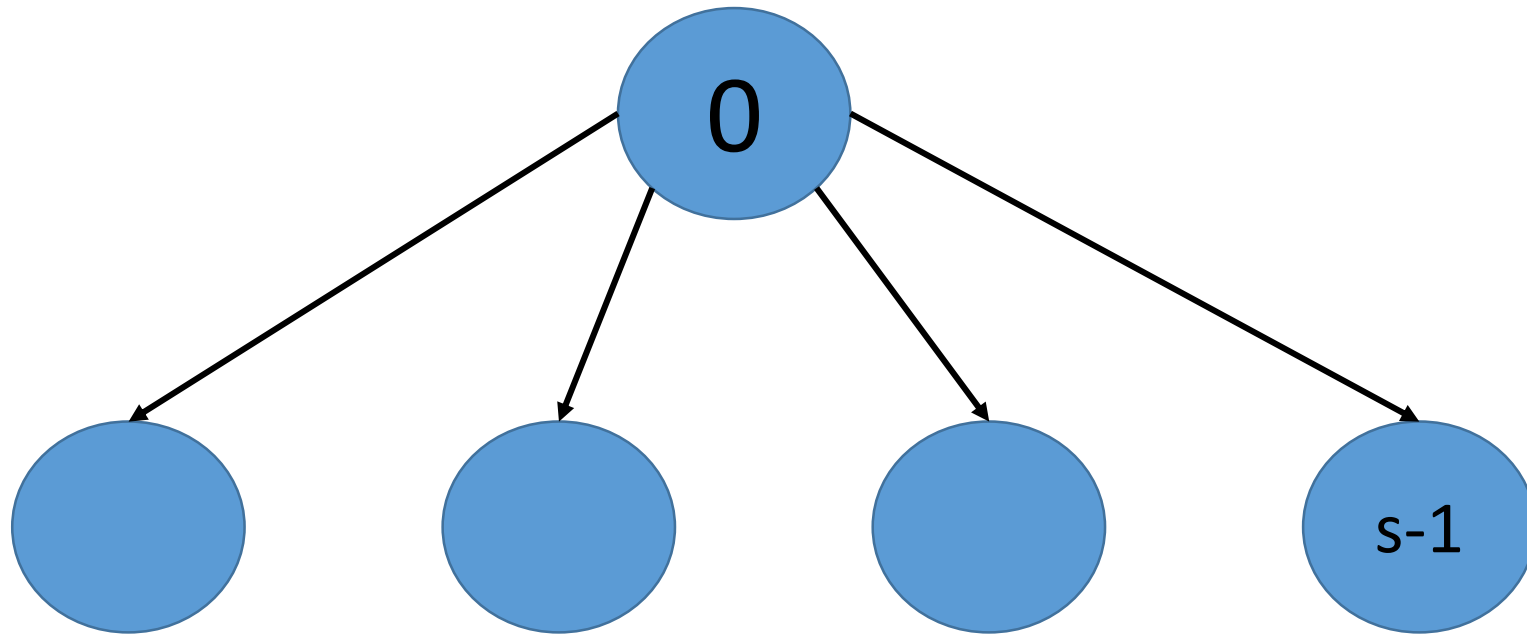
The Case of the Missing
Supercomputer
Performance: Achieving
Optimal Performance on
the 8,192 Processors of
ASCI Q, IEEE CLUSTER 2010



Optional Reading

A Quantitative Analysis of OS Noise, IPDPS 2011

One-to-many Sends



One-to-many Sends

```
sTime = MPI_Wtime();  
if (myrank == 0)  
    for (int r=1; r<size; r++)  
        MPI_Send (buf, count, MPI_INT, r, r, MPI_COMM_WORLD);  
else  
    MPI_Recv (buf, count, MPI_INT, 0, myrank, MPI_COMM_WORLD, &status);  
eTime = MPI_Wtime();  
time = eTime - sTime;  
  
// obtain max time  
MPI_Reduce (&time, &maxTime, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);  
if (!myrank) printf ("%lf\n", maxTime);
```

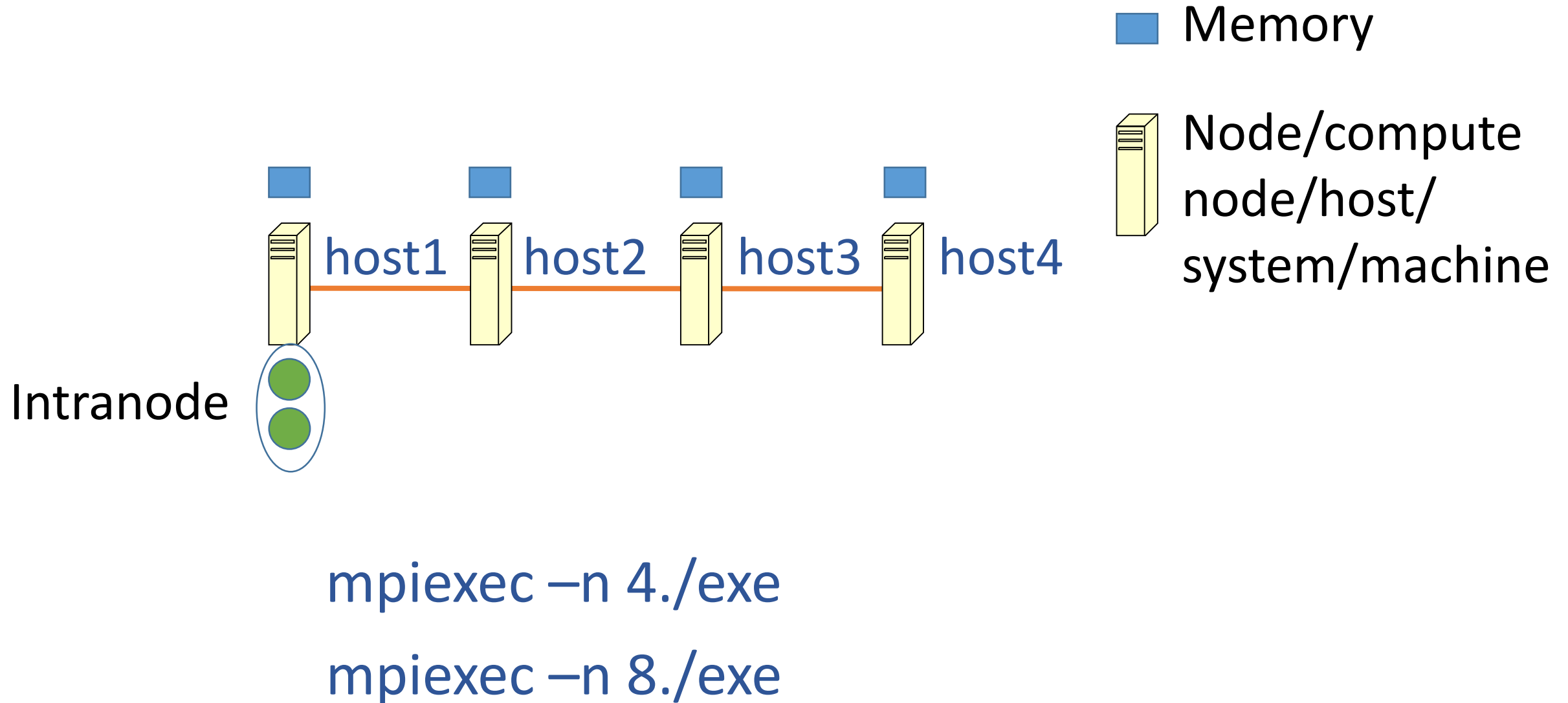
Timing

```
class $ for i in `seq 1 5`; do mpirun -np 4 ./onetomanyseeds 100 ; done
0.000032
0.000014
0.000013
0.000013
0.000012
class $ for i in `seq 1 5`; do mpirun -np 4 ./onetomanyseeds 1000000 ; done
0.002296
0.002655
0.002453
0.002084
0.002210
class $ for i in `seq 1 5`; do mpirun -np 8 ./onetomanyseeds 1000000 ; done
0.004122
0.005187
0.004641
0.004474
0.004451
```



Variability

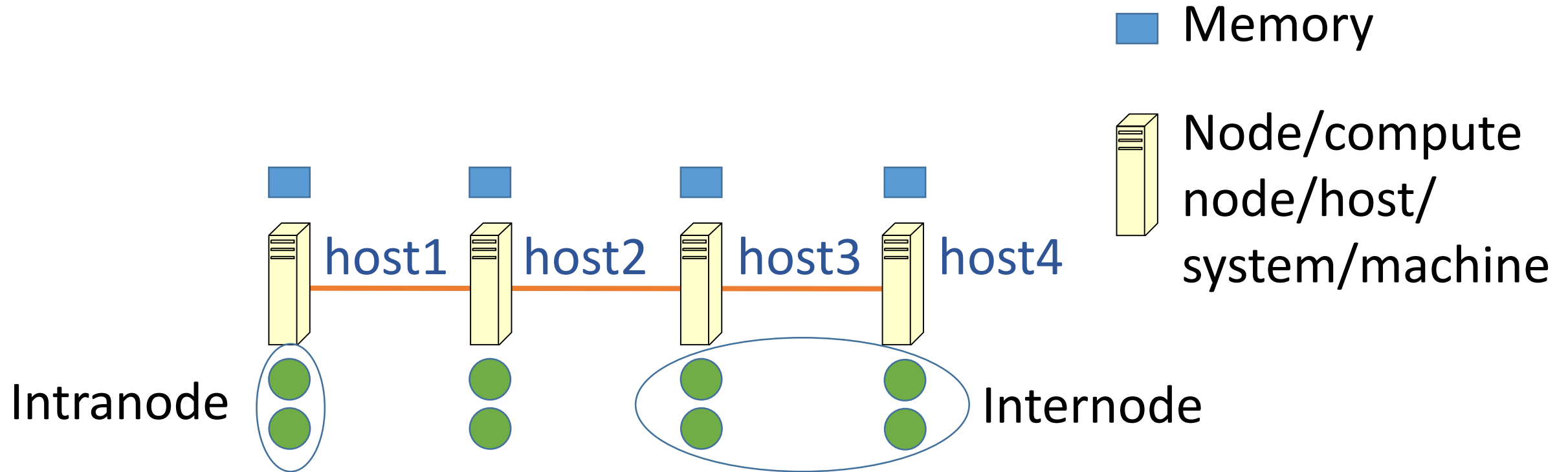
Execution Time Variability



Output

```
class $ for i in `seq 1 5`; do mpirun -np 8 ./onetomanyseeds 1000000 ; done
0.004122
0.005187
0.004641
0.004474
0.004451
class $ for i in `seq 1 5`; do mpirun -np 8 -hosts csews1:2,csews10:2,csews20:2,csews30:2 ./onetomanyseeds 1000000 ; done
0.205539
0.215736
0.217059
0.214883
0.208585
class $ █
```

Execution Time Variation

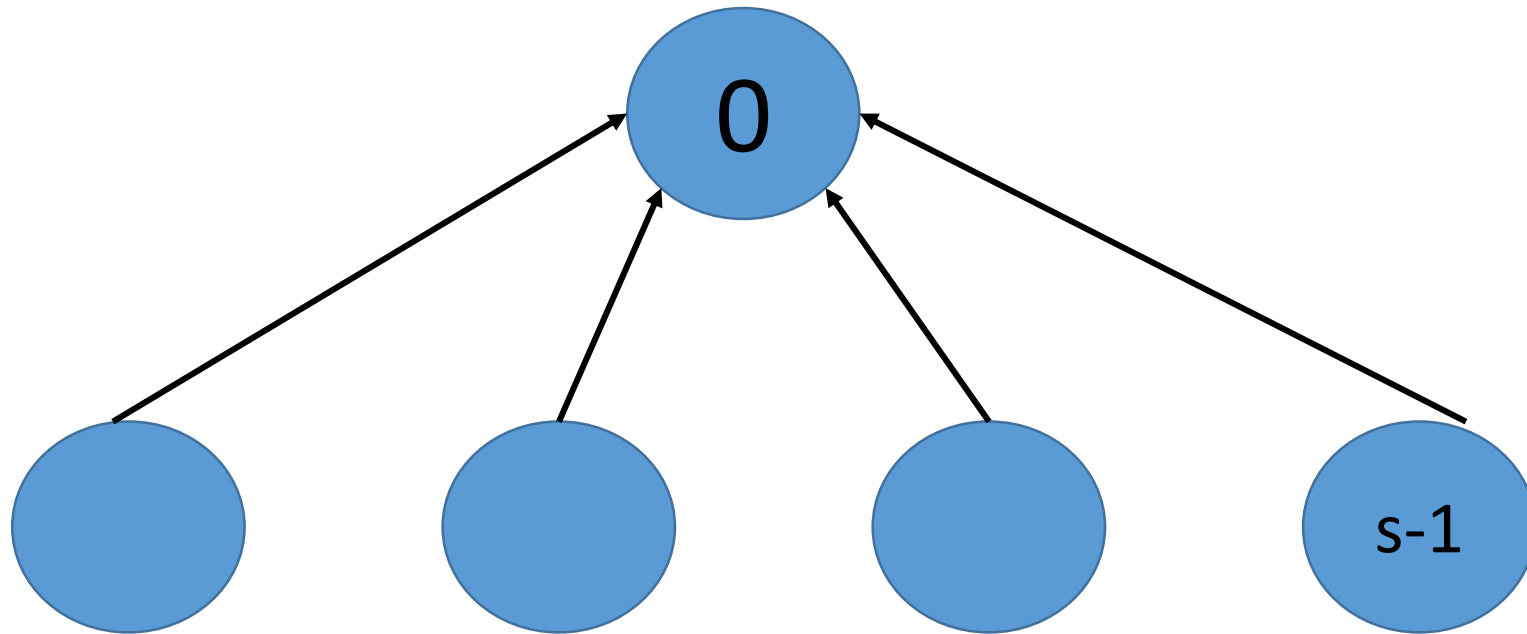


`mpixec -n 8 ./exe`

Intraconnect is faster typically

`mpixec -n 8 -hosts host1:2,host2:2,host3:2,host4:2 ./exe`

Many-to-one Sends



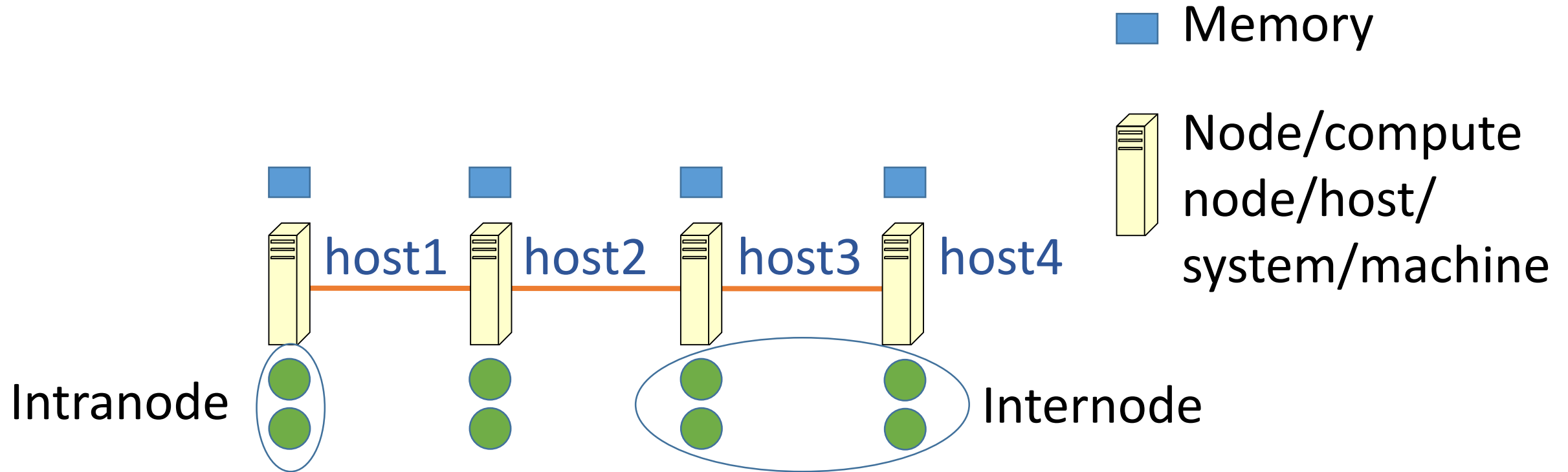
Many-to-one Sends

```
if (myrank)
{
    MPI_Send(arr, 20, MPI_INT, 0, myrank, MPI_COMM_WORLD);
}
else
{
    int count, recvarr[size][20];
    for (int i=1; i<size; i++)
    {
        MPI_Recv(recvarr[i], 20, MPI_INT, i, i, MPI_COMM_WORLD, &status); // slower
        printf("Rank %d of %d received from rank %d with tag %d\n", myrank, size, status.MPI_SOURCE, status.MPI_TAG);
    }
}
```

MPI_Send (Blocking send)

MPI_Recv (Blocking receive)

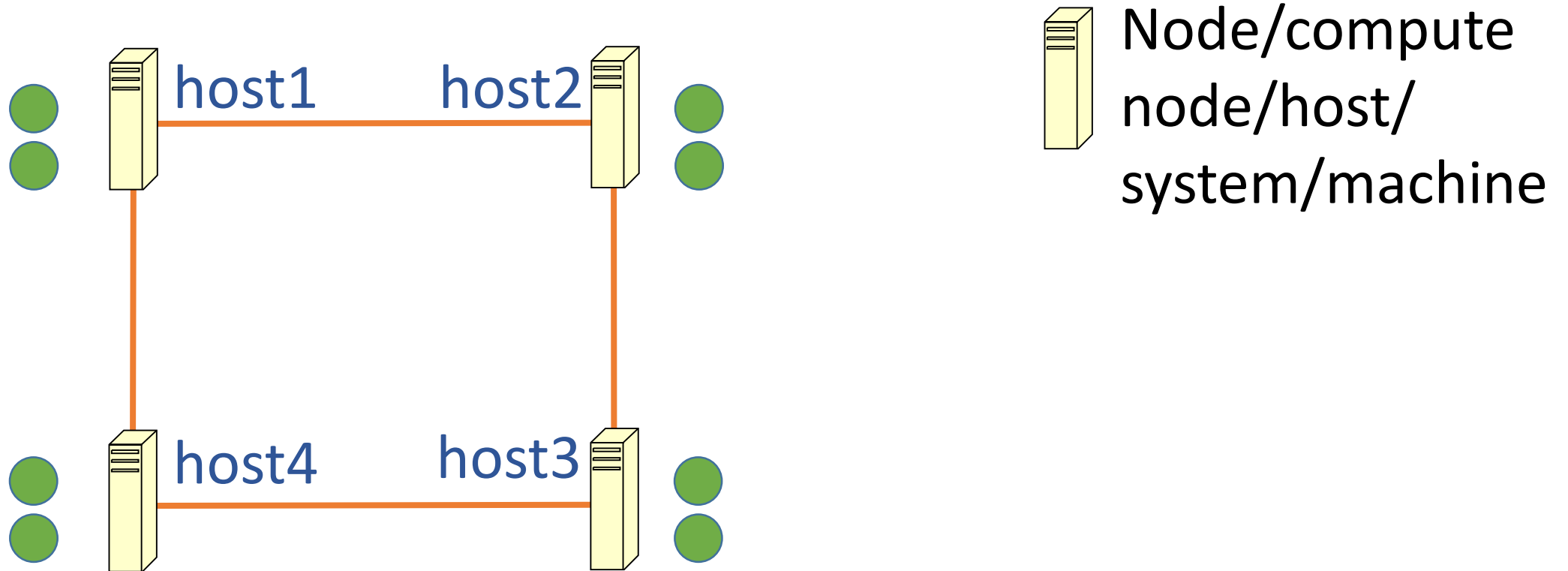
MPI Program Execution Order



```
mpirun -n 8 -hosts host1,host2,host3,host4 ./exe
```

```
mpirun -n 8 -hosts host1:2,host2:2,host3:2,host4:2 ./exe
```

MPI Program Execution Order



MPI_ANY_*

int MPI_Recv (void *buf, int count, MPI_Datatype datatype, int source, int tag, MPI_Comm comm, MPI_Status *status)

- MPI_ANY_SOURCE
 - Receiver may specify wildcard value for source
- MPI_ANY_TAG
 - Receiver may specify wildcard value for tag

Receive Out-of-order

a send operation
must specify a unique
receiver

```
if (myrank)
{
    MPI_Send(arr, 20, MPI_INT, 0, myrank, MPI_COMM_WORLD);
}
else
{
    int count, recvarr[size][20];
    for (int i=1; i<size; i++)
    {
        //MPI_Recv(recvarr[i], 20, MPI_INT, i, i, MPI_COMM_WORLD, &status); // may be slower
        MPI_Recv(recvarr[i], 20, MPI_INT, MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &status);
        printf("Rank %d of %d received from rank %d with tag %d\n", myrank, size, status.MPI_SOURCE, status.MPI_TAG);
    }
}
```

Rectify the receive
buffer location

No specific
order

Receive Out-of-order

```
if (myrank)
{
    MPI_Send(arr, 20, MPI_INT, 0, myrank, MPI_COMM_WORLD);
}
else
{
    int count, recvarr[size][20];
    for (int i=1; i<size; i++)
    {
        //MPI_Recv(recvarr[i], 20, MPI_INT, i, i, MPI_COMM_WORLD, &status); // may be slower
        MPI_Recv(recvarr[i], 20, MPI_INT, MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &status);
        printf("Rank %d of %d received from rank %d with tag %d\n", myrank, size, status.MPI_SOURCE, status.MPI_TAG);
    }
}
```

This may result in similar performance if MPI implementation performs buffering at the receiver side

- Design MPI implementation-agnostic code
- Design network-agnostic code
- Design configuration-agnostic code

Sum of Squares of N numbers

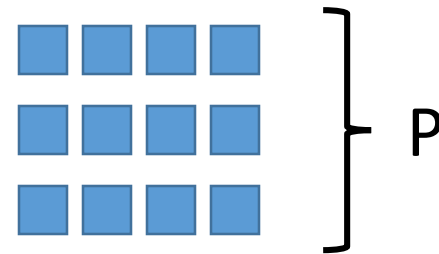
Serial

```
for i = 1 to N  
  sum += a[i] * a[i]
```



Parallel

```
for i = 1 to N/P  
  sum += a[i] * a[i]  
collate result
```



Sum of Squares



```
for i = 1 to N/P  
sum += a[i] * a[i]
```

Rank = 0

```
for i = 1 to N/P  
sum += a[i] * a[i]
```

Rank = 1

```
for i = 1 to N/P  
sum += a[i] * a[i]
```

Rank = 2

```
for i = 1 to N/P  
sum += a[i] * a[i]
```

Rank = 3

SIMD Parallelism

```
// Assume the large array is present in all ranks

for ( ; ; )
    localsum += a[i] * a[i]
// Collect localsum, add up at one of the ranks
if (!rank)
    recv from all ranks (...)
else
    send (0)
```